The result is truncated rather than rounded, so `s` will have a value of 54 in this example.

Durations support arithmetic, so you can add and subtract durations to get new durations or multiply or divide by a constant of the underlying representation type (the first template parameter). Thus `5*seconds(1)` is the same as `seconds(5)` or `minutes(1) –seconds(55)`. The count of the number of units in the duration can be obtained with the `count()` member function. Thus `std::chrono::milliseconds(1234).count()` is 1234.

Duration-based waits are done with instances of `std::chrono::duration<>`. For example, you can wait for up to 35 milliseconds for a future to be ready:

```
std::future<int> f=std::async(some_task);
if(f.wait_for(std::chrono::milliseconds(35))==std::future_status::ready)
    do_something_with(f.get());
```

The wait functions all return a status to indicate whether the wait timed out or the waited-for event occurred. In this case, you're waiting for a future, so the function returns `std::future_status::timeout` if the wait times out, `std::future_status::ready` if the future is ready, or `std::future_status::deferred` if the future's task is deferred. The time for a duration-based wait is measured using a steady clock internal to the library, so 35 milliseconds means 35 milliseconds of elapsed time, even if the system clock was adjusted (forward or back) during the wait. Of course, the vagaries of system scheduling and the varying precisions of OS clocks means that the actual time between the thread issuing the call and returning from it may be much longer than 35 ms.

With durations under our belt, we can now move on to time points.

4.3.3. Time points

The time point for a clock is represented by an instance of the `std::chrono::time_point<>` class template, which specifies which clock it refers to as the first template parameter and the units of measurement (a specialization of `std::chrono::duration<>`) as the second template parameter. The value of a time point is the length of time (in multiples of the specified duration) since a specific point in time called the *epoch* of the clock. The epoch of a clock is a basic property but not something that's directly available to query or specified by the C++ Standard. Typical epochs include 00:00 on January 1, 1970 and the instant when the computer running the application booted up. Clocks may share an epoch or have independent epochs. If two clocks share an epoch, the `time_pointtypedef` in one class may specify the other as the clock type associated with the `time_point`. Although you can't find out when the epoch is, you *can* get the `time_since_epoch()` for a given `time_point`. This member function returns a duration value specifying the length of time since the clock epoch to that particular time point.

For example, you might specify a time point as `std::chrono::time_point<std::chrono::system_clock, std::chrono::minutes>`. This would hold the time relative to the system clock but measured in minutes as opposed to the native precision of the system clock (which is typically seconds or less).